

Bridge Design Pattern

Category: Structural | **GoF Reference:** Gang of Four — Design Patterns (1994), p. 151

Table of Contents

1. [Intent](#)
 2. [The Problem It Solves — Class Explosion](#)
 3. [When to Use](#)
 4. [Structure — ASCII UML & The Players](#)
 5. [Package Structure Explained](#)
 6. [Code Walkthrough — Every File, Every Line](#)
 7. [Why These Design Decisions](#)
 8. [Execution Flow Trace](#)
 9. [Real-World Use Cases](#)
 10. [Bridge vs Adapter](#)
 11. [Expected Output](#)
 12. [How to Run](#)
-

1. Intent

The Bridge pattern **separates an abstraction from its implementation** so that the two can vary independently.

The word "bridge" is deliberate: the pattern literally builds a bridge between two independent class hierarchies — the abstraction hierarchy and the implementation hierarchy — and connects them through **composition rather than inheritance**. Each hierarchy can grow and change without affecting the other.

In classical inheritance, you extend a base class to add behavior, and you extend it again to add a variant of that behavior. This works for one dimension of variation. The moment you have *two* orthogonal dimensions (for example, *shapes* and *colors*, or *UI controls* and *rendering engines*, or *payment flows* and *payment gateways*), inheritance starts generating a combinatorial explosion of subclasses. Bridge cuts that explosion down by making one hierarchy *hold a reference* to the other instead of inheriting from it.

The core idea: **"Prefer composition over inheritance."** Bridge is one of the purest expressions of this principle in the GoF catalog.

2. The Problem It Solves — Class Explosion

Suppose you are building a drawing library. You have shapes and colors. Without Bridge, you model every combination as its own class:

```
RedTriangle
BlueTriangle
GreenTriangle
RedCircle
BlueCircle
GreenCircle
RedSquare
BlueSquare
GreenSquare
```

With 3 shapes and 3 colors you already have **9 classes**. Add a fourth color (Yellow) and you must add 3 more classes. Add a fourth shape (Pentagon) and you must add 4 more classes. The formula is:

```
Total classes = N shapes × M colors
```

At 10 shapes and 10 colors: **100 classes**. At 20 shapes and 20 colors: **400 classes**. The codebase grows quadratically. Each new class mostly duplicates logic from its siblings. Tests multiply. Every refactor touches dozens of files.

Bridge collapses this to:

```
Total classes = N shapes + M colors
```

At 10 shapes and 10 colors: **20 classes**. At 20 shapes and 20 colors: **40 classes**. The codebase grows linearly. Adding a new color is one file. Adding a new shape is one file. Neither touches the other.

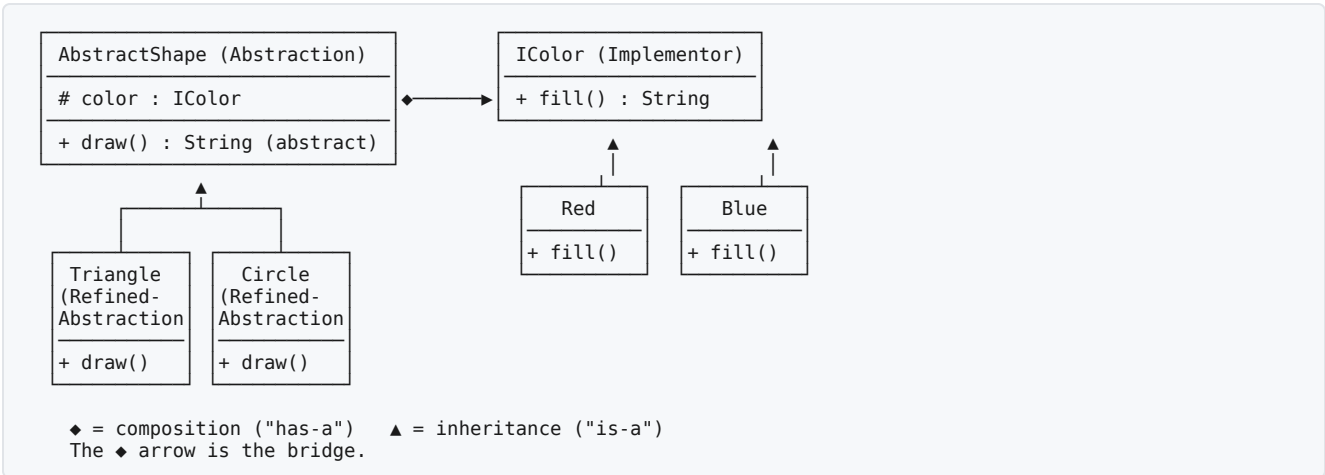
The mechanism: instead of `RedTriangle` extends `Triangle`, you write `Triangle` with a field of type `IColor`. The triangle delegates color-related behavior to whichever `IColor` instance it holds at runtime. This is the bridge. This module demonstrates exactly that mechanism with two shapes (`Triangle`, `Circle`) and two colors (`Red`, `Blue`) — four combinations produced from four classes, not four dedicated classes.

3. When to Use

Use the Bridge pattern when:

1. **You want to avoid a permanent binding between abstraction and implementation.** If the implementation should be selectable or switchable at runtime — for example, choosing a database driver at startup or selecting a rendering backend based on the OS — Bridge lets you inject any conforming implementation without changing the abstraction.
2. **Both the abstraction and its implementation should be extensible through subclassing, independently.** If you have two separate reasons a class might grow (e.g., more shapes vs. more colors), those two reasons belong in two separate hierarchies, not one tangled one.
3. **Changes to the implementation should not impact client code.** Clients program to the abstraction (`AbstractShape`). The `IColor` interface can change its internals, gain new implementations, or be swapped entirely — clients never recompile or change.
4. **You have a class explosion caused by multiple dimensions of variation.** Whenever you find yourself naming classes `XYZ` where X varies along one axis and Z varies along another (e.g., `MySQLUserRepo`, `PostgresUserRepo`, `MySQLOrderRepo`, `PostgresOrderRepo`), Bridge is the remedy.
5. **You need to share an implementation among multiple objects and this fact should be hidden from the client.** A single `IColor` instance can be shared by many shapes. The client uses `AbstractShape`, never knowing whether the color object is shared or exclusive.
6. **You are working across platform boundaries.** Bridge is especially natural when the abstraction lives in portable, platform-independent code (your business logic) and the implementation lives in platform-specific code (OS calls, vendor SDKs, hardware drivers).

4. Structure — ASCII UML & The Players



The players:

Role	Class in this module	Responsibility
Abstraction	<code>AbstractShape</code>	Defines the high-level interface (<code>draw()</code>); holds the <code>IColor</code> implementor reference
RefinedAbstraction	<code>Triangle</code> , <code>Circle</code>	Extend the abstraction; each supplies its own <code>draw()</code> logic while delegating color to the held <code>IColor</code>
Implementor	<code>IColor</code>	Defines the low-level interface (<code>fill()</code>) the abstraction delegates to
ConcreteImplementor	<code>Red</code> , <code>Blue</code>	Provide a concrete rendering of the implementor interface

The critical structural point: `AbstractShape` *contains* `IColor` through composition (the `♦` arrow). This is the bridge. `Triangle` does not extend `Red`; it holds an `IColor` and calls `color.fill()` when drawing. The shape hierarchy (`AbstractShape` → `Triangle` / `Circle`) and the color hierarchy (`IColor` → `Red` / `Blue`) are two genuinely independent hierarchies connected only through that one field — this is real two-hierarchy Bridge, not a single-hierarchy Strategy dressed up as one.

5. Package Structure Explained

```
com.design.patterns.bridge
├── BridgeDesignPattern.java      (driver / main class)
├── contract/
│   ├── IColor.java              (Implementor interface)
│   └── AbstractShape.java       (Abstraction abstract class)
└── concrete/
    ├── Red.java                 (ConcreteImplementor)
    ├── Blue.java                (ConcreteImplementor)
    ├── Triangle.java            (RefinedAbstraction)
    └── Circle.java              (RefinedAbstraction)
```

Why `contract` holds both interfaces?

The `contract` package groups the *stable, defining contracts* of the pattern — the parts that change least. `IColor` is the implementor interface; `AbstractShape` is the abstraction. Both define APIs (contracts) that the rest of the codebase depends on. Placing them together signals: "these are the anchors; extensions live elsewhere." Client code that wants to program against abstractions only needs to import from `contract`.

Why `concrete` is a sub-package of `contract`?

The `concrete` sub-package signals: "these are concrete realizations of the contracts defined one level up." Keeping them inside `contract` preserves locality — you can find the interface and all its implementations by navigating a single package tree — while still separating the stable API (`contract`) from the volatile implementations (`contract.concrete`). The sub-package structure also mirrors the dependency direction: `concrete` depends on `contract`, never the reverse.

Why are both `Red` / `Blue` (color implementations) and `Triangle` / `Circle` (shape implementations) in the same `concrete` package?

In this pedagogical implementation, both dimensions are concrete extensions, so they share one `concrete` package. In a production codebase you would likely split them further: `contract.colors` and `contract.shapes`, each holding their respective implementations.

6. Code Walkthrough — Every File, Every Line

6.1 `IColor.java` — The Implementor Interface

```
package com.design.patterns.bridge.contract;
```

Declares that this type belongs to the `contract` package — the stable API layer. All classes that depend on the `IColor` contract import from here.

```
public interface IColor {
```

Declares `IColor` as a Java interface, not a class. An interface enforces a contract without providing implementation, making it the correct choice for the implementor role. Any class that wants to be a color in this system must implement this interface. The name `IColor` is intentional: it describes the *concept*, not any specific color.

```
    String fill();
```

The single method of the implementor interface. It takes no parameters and returns a `String` representing the color's name. The method is named `fill` because it conceptually "fills" a shape with a color. Returning a `String` (rather than printing directly or returning `void`) makes the method pure and testable — it produces output that callers can use, log, or assert against.

```
}
```

Closes the interface declaration. It is intentionally minimal — one method, one responsibility, following the Interface Segregation Principle.

6.2 AbstractShape.java — The Abstraction

```
package com.design.patterns.bridge.contract;
```

Same package as `IColor`. Both contracts live together in `com.design.patterns.bridge.contract`.

```
public abstract class AbstractShape {
```

`AbstractShape` is an abstract class, not an interface — a deliberate choice discussed in Section 7. It is partially implemented: it holds the `color` field and the constructor, but leaves `draw()` for subclasses to complete. The `abstract` keyword prevents instantiation of `AbstractShape` directly.

```
    protected final IColor color;
```

This single field is the **bridge**. `AbstractShape` does not extend `IColor`; it *holds* an `IColor`. `protected` allows subclasses (`Triangle`, `Circle`) to read `color` directly without needing a getter. `final` ensures the reference cannot be reassigned after construction — a shape's color identity is fixed at birth. The field type is the `IColor` interface, not a concrete type: `AbstractShape` knows only that it holds something that can `fill()`, not whether that something is `Red`, `Blue`, or any other color.

```
    protected AbstractShape(IColor color) {
```

The constructor accepts an `IColor` and stores it. It is `protected` so that only subclasses and classes in the same package can call it — external code cannot instantiate `AbstractShape` directly (nor would they want to, since it is abstract). The parameter name `color` shadows the field name, resolved by `this.color = color` on the next line.

```
        this.color = color;
```

Assigns the injected `IColor` implementation to the field. This is **constructor injection** — the dependency (`IColor`) is injected through the constructor rather than created inside the class. This makes `AbstractShape` independent of any specific color implementation.

```
    abstract public String draw();
```

Declares the abstract method that all concrete shapes must implement. The `String` return type (parallel to `IColor.fill()`) keeps the method pure. Every shape has its own way of describing itself, but all shapes expose the same signature — `draw()` — so client code can call it on any `AbstractShape` without knowing its concrete type.

```
}
```

Closes the abstract class.

6.3 Red.java — Concrete Implementor

```
package com.design.patterns.bridge.contract.concrete;
```

Located in the `concrete` sub-package. This is a concrete class — an implementation — so it lives below the `contract` layer.

```
import com.design.patterns.bridge.contract.IColor;
```

Imports the `IColor` interface that this class implements. The dependency direction is correct: `concrete` depends on `contract`, not the other way around.

```
public class Red implements IColor {
```

`Red` is a concrete class that provides one implementation of the `IColor` contract. `implements IColor` makes `Red` a valid substitution wherever an `IColor` is expected — including the `AbstractShape` constructor.

```
@Override
public String fill() {
    return "Red";
}
```

The concrete implementation of `fill()`. It returns the string `"Red"`. In a real application, this might return an ANSI color code, an RGB hex value, or a localized color name.

```
}
```

Closes the class.

6.4 `Blue.java` — Concrete Implementor

```
package com.design.patterns.bridge.contract.concrete;
```

Same package as `Red`. Both concrete colors live together.

```
import com.design.patterns.bridge.contract.IColor;
```

Same import as `Red`. Both implementors depend on the same `IColor` interface.

```
public class Blue implements IColor {

    @Override
    public String fill() {
        return "Blue";
    }
}
```

A second concrete implementor, structurally identical to `Red` except for the returned string. `Blue` and `Red` are siblings — both implement `IColor`, both are interchangeable anywhere an `IColor` is needed, and both return their name with the same capitalization (`"Red"`, `"Blue"`) so `AbstractShape.draw()` output is consistent regardless of which color is plugged in. Neither knows about the other. Neither knows about `AbstractShape`.

6.5 `Triangle.java` — Refined Abstraction

```
package com.design.patterns.bridge.contract.concrete;
```

`Triangle` is a concrete shape — a refined abstraction — so it lives in `concrete` alongside the concrete colors.

```
import com.design.patterns.bridge.contract.IColor;
import com.design.patterns.bridge.contract.AbstractShape;
```

`Triangle`'s constructor needs `IColor` as a parameter type to pass to `super()`, and it needs `AbstractShape` because it extends it.

```
public class Triangle extends AbstractShape {
```

`Triangle` is a concrete, instantiable class that extends the abstract `AbstractShape`. It is the "Refined Abstraction" in Bridge terminology: it extends the abstraction by providing a concrete `draw()` for triangles specifically.

```
    public Triangle(IColor color) {
        super(color);
    }
```

The constructor accepts an `IColor` argument and immediately delegates to `AbstractShape`'s protected constructor via `super(color)`, which stores the reference in the inherited protected final `IColor color` field. The constructor is `public` because external code (the driver) needs to instantiate `Triangle` directly. The parameter type is `IColor` (the interface), not `Red` or `Blue`, so `Triangle` is decoupled from any specific color — `new Triangle(new Red())`, `new Triangle(new Blue())`, or `new Triangle(anyColorYouInvent)` all work unchanged. `super(color)` is mandatory here: Java requires a call to the superclass constructor as the first statement in a subclass constructor, and `AbstractShape` has no no-arg constructor.

```
@Override
public String draw() {
    return "Triangle shape with color : " + color.fill();
}
```

The concrete implementation of the abstract method declared in `AbstractShape`. This line is the bridge **in action**: `Triangle` calls `color.fill()` — it delegates the color-related behavior to whichever `IColor` implementor it was given at construction time, without knowing if it is `Red`, `Blue`, or anything else. The concatenation builds the complete result from `Triangle`'s own part (`"Triangle shape with color : "`) and the color's part (`color.fill()`).

```
}
```

Closes the class.

6.6 `Circle.java` — Refined Abstraction

```
package com.design.patterns.bridge.contract.concrete;

import com.design.patterns.bridge.contract.IColor;
import com.design.patterns.bridge.contract.AbstractShape;

public class Circle extends AbstractShape {
    public Circle(IColor color) {
        super(color);
    }

    @Override
    public String draw() {
        return "Circle shape with color : " + color.fill();
    }
}
```

`Circle` is the second `RefinedAbstraction`, structurally identical to `Triangle` in every respect except the literal shape name in the output string (`"Circle shape with color : "` instead of `"Triangle shape with color : "`) and its own class name. This is exactly the point of Bridge: adding a new shape costs one small class that repeats the same three-line pattern (constructor delegates to `super(color)`, `draw()` concatenates its own label with `color.fill()`) — it required zero changes to `AbstractShape`, `IColor`, `Red`, or `Blue`. Independently, `Circle` and `Triangle` can each be combined with `Red` or `Blue` without either shape or either color ever referencing the other's class.

6.7 `BridgeDesignPattern.java` — The Driver

```
package com.design.patterns.bridge;
```

The driver lives in the root package of the module, one level above `contract`. It is the entry point, not a pattern participant.

```
import com.design.patterns.bridge.contract.AbstractShape;
import com.design.patterns.bridge.contract.concrete.Blue;
import com.design.patterns.bridge.contract.concrete.Circle;
import com.design.patterns.bridge.contract.concrete.Red;
import com.design.patterns.bridge.contract.concrete.Triangle;
```

The driver imports the `AbstractShape` abstraction (the left side of the bridge) plus all four concrete leaf classes it needs to construct: the two `RefinedAbstractions` (`Triangle`, `Circle`) and the two `ConcreteImplementors` (`Red`, `Blue`). The driver is the one place in the module allowed to know about every concrete class — once objects are built, the rest of the code works through the `AbstractShape` and `IColor` interfaces only.

```
public class BridgeDesignPattern {
```

The driver class, named after the pattern. It is a plain class with a static `main` — no inheritance, no pattern participation.

```
    public static void main(String[] args) {
```

The standard JVM entry point. This is the only method in the class.

```
        System.out.println("Bridge Design Pattern");
```

Prints the pattern name as a header, confirming the program launched and identifying the output visually.

```
AbstractShape[] shapes = { new Triangle(new Red()), new Triangle(new Blue()),  
    new Circle(new Red()), new Circle(new Blue()) };
```

Builds an array of four `AbstractShape` references, one for every combination of the two `RefinedAbstractions` and the two `ConcreteImplementors`. Each element assembles a bridge on the spot: `new Red()` or `new Blue()` creates the implementor half, and wrapping it in `new Triangle(...)` or `new Circle(...)` creates the abstraction half around it. The array is declared as `AbstractShape[]`, not `Triangle[]` or `Object[]` — every element is stored and later used purely as an abstraction, even though the driver had to name the concrete types to construct them. This one array is the clearest demonstration in the module that the two hierarchies multiply combinatorially (2 shapes × 2 colors = 4 objects) while the source still only needed one class per shape and one class per color (2 + 2 = 4 classes).

```
for (AbstractShape shape : shapes) {  
    System.out.println(shape.draw());  
}
```

Iterates the array using only the `AbstractShape` abstraction. Each call to `shape.draw()` is a polymorphic dispatch to whichever concrete `draw()` the runtime type provides (`Triangle.draw()` or `Circle.draw()`), which in turn calls `color.fill()` on whichever `IColor` that shape was constructed with (`Red.fill()` or `Blue.fill()`). The loop body never mentions `Triangle`, `Circle`, `Red`, or `Blue` by name — it is written entirely against `AbstractShape`, and would be unchanged if a fifth or sixth combination were appended to the array above.

```
    }  
}
```

Closes the method and the class.

7. Why These Design Decisions

Why `protected final IColor color`?

Three separate modifiers, each with a distinct purpose:

- **protected**: The `color` field needs to be accessible in `Triangle.draw()` and `Circle.draw()` without a getter method. Since both are subclasses of `AbstractShape`, `protected` grants access. Making it `private` would force a getter, adding boilerplate. Making it `public` would expose the internal implementor reference to all client code, breaking encapsulation.
- **final**: A shape's color is set at construction and should not change. `final` enforces this at compile time — no code inside `AbstractShape` or any subclass can reassign `color` after the constructor returns. This makes `AbstractShape` instances easier to reason about: once built, their color never changes.
- **IColor interface type, not concrete**: `AbstractShape` holds a reference of type `IColor`, not `Red` or `Blue`. This is what makes the bridge work — `AbstractShape` is decoupled from all specific colors. Any `IColor` implementation can be passed at construction time, now or in the future, without modifying `AbstractShape`.

Why `abstract class AbstractShape` instead of `interface AbstractShape`?

An interface cannot hold instance fields or provide constructor logic. The bridge requires `AbstractShape` to store an `IColor` reference in `protected final IColor color` and initialize it via `protected AbstractShape(IColor color)`. Neither is possible in an interface.

An abstract class is the right tool when you need to: 1. Define abstract methods that subclasses must implement (`draw()`). 2. Provide concrete state shared by all subclasses (`color` field). 3. Enforce an initialization protocol via a constructor (`this.color = color`).

An interface would force each concrete shape (`Triangle`, `Circle`, and any future shape) to independently declare and initialize its own `color` field, duplicating the bridge setup in every one. The abstract class centralizes that boilerplate once.

Why `super(color)` in the `Triangle` and `Circle` constructors?

`AbstractShape` declares no no-arg constructor. When you subclass a class that has no default constructor, Java requires that your subclass constructor explicitly call a superclass constructor as its first statement. `super(color)` invokes `AbstractShape(IColor color)`, which stores the `IColor` reference.

Beyond the mechanical requirement, `super(color)` is correct by design: the bridge connection (`this.color = color`) belongs in the base abstraction (`AbstractShape`), not in each refined abstraction. Every shape needs a color stored the same way. Centralizing that in `AbstractShape`'s constructor and delegating to it via `super()` is the DRY (Don't Repeat

Yourself) expression of the pattern — it is also exactly why adding `Circle` next to `Triangle` cost three lines instead of duplicating field declarations and null-checks.

Why does `String fill()` return a value instead of `void`?

A `void fill()` would be forced to produce output as a side effect — either printing directly (`System.out.println`) or writing to some shared mutable state. Side-effecting implementors are:

- **Harder to test:** You cannot assert on a return value; you must intercept stdout or check shared state.
- **Less composable:** `Triangle.draw()` and `Circle.draw()` could not build a string that *includes* the color; they could only trigger the color to print separately, losing the ability to compose the output.
- **Inflexible:** The caller (the shape) loses control over what happens with the color information — it cannot log it, return it to a higher caller, or format it differently.

Returning `String` keeps `fill()` a **pure function**: given no input, it returns a predictable value with no side effects. Each shape's `draw()` composes the full output string (its own label plus `color.fill()`) and returns it, leaving the decision of what to do with that string to the caller. The driver (`BridgeDesignPattern`) is the only place that decides to print it.

8. Execution Flow Trace

Here is a step-by-step trace of exactly what happens from the moment the JVM calls `main()` to the moment the program exits:

```
1. JVM calls BridgeDesignPattern.main(String[])

2. System.out.println("Bridge Design Pattern")
   → Prints: "Bridge Design Pattern"

3. AbstractShape[] shapes = { ... } – four bridges are assembled, left to right:

   a) new Triangle(new Red())
      → new Red() : allocates a Red instance (no fields, no logic beyond Object()).
      → new Triangle(IColor) is called with that Red instance.
      → Inside Triangle(): super(color) → AbstractShape(IColor color): this.color = color.
      → Returns a Triangle reference holding a Red.

   b) new Triangle(new Blue()) – same sequence, holding a Blue instead of a Red.
   c) new Circle(new Red()) – same sequence, constructing a Circle instead of a Triangle.
   d) new Circle(new Blue()) – same sequence, a Circle holding a Blue.

   → All four references are stored in the shapes array, typed as AbstractShape[].

4. for (AbstractShape shape : shapes) – the loop runs once per array element:

   Iteration 1 – shape is the Triangle holding Red:
   → shape.draw() dispatches polymorphically to Triangle.draw().
   → Inside Triangle.draw(): color.fill() dispatches to Red.fill() → returns "Red".
   → Concatenation produces: "Triangle shape with color : Red"
   → System.out.println(...) prints it.

   Iteration 2 – shape is the Triangle holding Blue:
   → Triangle.draw() calls Blue.fill() → "Blue".
   → Prints: "Triangle shape with color : Blue"

   Iteration 3 – shape is the Circle holding Red:
   → Circle.draw() calls Red.fill() → "Red".
   → Prints: "Circle shape with color : Red"

   Iteration 4 – shape is the Circle holding Blue:
   → Circle.draw() calls Blue.fill() → "Blue".
   → Prints: "Circle shape with color : Blue"

5. main() returns. JVM exits.
```

The bridge crossing happens inside every call to `draw()`: `Triangle/Circle` call across to `Red/Blue` purely through the `IColor` interface. Neither shape hierarchy nor color hierarchy references the other's concrete class — they communicate only through `IColor.fill()`. You could add a `Green` implementor or a `Square` refined abstraction and every existing line above would behave identically for the combinations that already exist.

9. Real-World Use Cases

9.1 GUI Frameworks — Rendering API vs. UI Controls

GUI toolkits like Java's AWT/Swing or Qt face a classic Bridge scenario. There are two independent axes of variation:

- **UI Controls:** Button, TextField, CheckBox, ComboBox, ...
- **Rendering Backends:** Windows GDI, macOS Quartz, Linux X11, OpenGL, ...

Without Bridge: `WindowsButton`, `MacButton`, `LinuxButton`, `WindowsTextField`, `MacTextField`, ... Dozens of classes, each mixing UI logic with platform rendering code.

With Bridge: `Button` holds a reference to a `Renderer` interface. `WindowsRenderer`, `MacRenderer`, `OpenGLRenderer` implement `Renderer`. Adding a new control type adds one class. Adding a new platform adds one renderer. The two hierarchies evolve independently.

9.2 Database Drivers — JDBC Abstraction vs. Vendor Implementation

Java's JDBC is a textbook Bridge. Application code uses `java.sql.Connection`, `Statement`, `ResultSet` — these are the abstraction layer. MySQL Connector/J, PostgreSQL JDBC, Oracle Thin Driver — these are the implementations. Your application never touches MySQL-specific classes; it holds a `Connection`. Switching databases is swapping the `Driver` registration at startup; no application code changes.

9.3 Logging Frameworks — Logger API vs. Appender

SLF4J is a Bridge. Application code calls `Logger.info(...)` — the abstraction. Logback, Log4j2, `java.util.logging` — these are implementations plugged in at runtime via the `LoggerFactory` binding. Your application compiles against `org.slf4j:slf4j-api`. Which backend runs is a deployment decision. Adding a new log destination (Splunk, Datadog) means adding a new Appender implementation; application code is untouched.

9.4 Payment Processing — Payment Flow vs. Payment Gateway

An e-commerce platform has two axes:

- **Payment flows:** `OneTimeCharge`, `Subscription`, `Installment`, `Refund`
- **Payment gateways:** `Stripe`, `PayPal`, `Razorpay`, `BankTransfer`

Without Bridge: `StripeOneTimeCharge`, `PayPalOneTimeCharge`, `RazorpaySubscription`, ... — N×M classes, each pairing a flow with a gateway.

With Bridge: `PaymentFlow` (abstraction) holds a `PaymentGateway` (implementor). `Subscription` extends `PaymentFlow` handles recurring logic; `StripeGateway` implements `PaymentGateway` handles Stripe API calls. Adding PayPal requires one class. Adding a Layaway flow requires one class. Neither touches the other.

9.5 Device Drivers — OS Abstraction vs. Hardware Implementation

Operating systems use Bridge to manage hardware. The kernel defines an abstract device interface: `read()`, `write()`, `ioctl()`. Hardware vendors implement that interface for their specific chips. A USB storage driver and a network card driver both conform to the same kernel interface. The OS "abstraction" (file system, network stack) calls the standard interface; hardware vendors provide the "implementors." Adding a new graphics card model requires a new driver implementation; the kernel is unaffected.

10. Bridge vs Adapter

These two patterns are frequently confused because both involve two class hierarchies communicating through a reference. The critical difference is *when* and *why* they are applied.

Dimension	Bridge	Adapter
Purpose	Design for extensibility upfront	Make two incompatible interfaces work together retroactively
When designed	Before implementation — part of the initial architecture	After the fact — one class already exists with an incompatible interface
Relationship	Abstraction owns the implementor reference from day one	Adapter wraps an existing class to satisfy a new interface
Direction	Both hierarchies designed together with the bridge in mind	Adapter is a one-way adapter: it translates to a pre-existing target
Intent	"These two things should vary independently"	"I need to use this existing class, but it doesn't match the interface I need"

Concrete example of the distinction:

- Bridge (this module): `AbstractShape` and `IColor` are designed together. `AbstractShape` is written knowing it will hold an `IColor`. The bridge is intentional from the start.
- Adapter: You have a legacy `LegacyColorProvider` class with a method `getColorName()`. You need it to work as an `IColor` (which requires `fill()`). You write a `LegacyColorAdapter` implements `IColor` that wraps

`LegacyColorProvider` and delegates `fill()` to `getColorName()`. You never changed `LegacyColorProvider`; you adapted it.

Think of it this way: Bridge is **architectural** (planned), Adapter is **tactical** (after-the-fact fix).

11. Expected Output

Running `BridgeDesignPattern.main()` produces exactly:

```
Bridge Design Pattern
Triangle shape with color : Red
Triangle shape with color : Blue
Circle shape with color : Red
Circle shape with color : Blue
```

Four lines after the header — one per element of the `shapes` array, in construction order: Triangle + Red, Triangle + Blue, Circle + Red, Circle + Blue. `Red.fill()` and `Blue.fill()` both return their name with the same capitalization ("Red", "Blue"), so the four output lines differ only in shape label and color label — nothing else varies.

To add a fifth combination, append one element to the `shapes` array in `BridgeDesignPattern.java`, e.g. `new Circle(new Red())` again, or introduce a new shape/color class and reference it there. No other file needs to change.

12. How to Run

This module is a Maven submodule of the `Design_Patterns` reactor (parent POM at `../pom.xml`, groupId `com.design.patterns`, artifactId `Bridge_Design_Pattern`). It targets Java 1.8 source/target via the parent's Spring Boot 2.7.7 configuration, but has no Spring runtime dependency of its own — `main()` is a plain JVM entry point.

Build:

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
```

Run:

```
/usr/lib/jvm/java-11-openjdk-amd64/bin/java -cp target/classes com.design.patterns.bridge.BridgeDesignPattern
```

(Any JDK 8+ works at runtime; JDK 11 is used above only because the reactor's default JDK breaks an unrelated Lombok-based sibling module. This module itself has no Lombok dependency.)